

FDTD Implementation Guide

Tools, Wave Injection, Measurement, and Full Code Walkthrough

EE2 Design Project — Imperial College London

What This Document Covers

Section 1: Every software tool you need and how to install it. Section 2: The complete Vivado project setup from scratch. Section 3: How wave injection works physically and in Verilog — hard source, soft source, multiple sources. Section 4: How to measure the wave — what the XADC sees, the lock-in pipeline, the probe ADC circuit. Section 5: Complete Verilog code for every FDTD module. Section 6: Complete Python control layer. Section 7: Testing and debugging strategy. Section 8: How everything connects on the Pynq board physically.

SECTION 1 — TOOLS

1. Tools You Need

1.1 Complete Tool List

Tool	Version	Purpose	Download
Vivado Design Suite	2022.2 or later	FPGA synthesis, implementation, bitstream generation	xilinx.com/support/download
Icarus Verilog	11.0+	Verilog simulation — fast, free, runs on Linux/Mac/Windows	iverilog.icarus.com
GTKWave	3.3+	Waveform viewer for simulation output — inspect signal values over time	gtkwave.sourceforge.net
Python 3	3.8+	Pynq control layer, numpy for signal processing	pre-installed on Pynq
Pynq library	2.7+	Python API for FPGA overlay loading, MMIO, DMA	pre-installed on Pynq
KiCad	7.0+	PCB design for probe circuit board	kicad.org
EasyEDA	Web browser	Simpler PCB design alternative, direct JLCPCB integration	easyeda.com
VS Code + Verilog extension	Latest	Text editor with Verilog syntax highlighting and linting	code.visualstudio.com
FileZilla or SCP	Any	Transfer files between your laptop and Pynq over SSH	filezilla-project.org
PuTTY (Windows) / Terminal	Any	SSH into Pynq from laptop	putty.org

1.2 Installing Vivado

Vivado is the Xilinx (AMD) FPGA design tool. It is free for Pynq-Z1 (Artix-7 / Zynq-7000 devices) under the WebPACK licence — no licence server needed.

1. Go to [xilinx.com/support/download](https://www.xilinx.com/support/download) and download Vivado ML Edition 2022.2 (or latest).
2. Run the installer. When asked which devices to support, select 'Zynq-7000' only — reduces install size from 100GB to ~30GB.

3. Select 'Vivado ML Standard' (free WebPACK licence covers Zynq-7000).
4. After installation, open Vivado and go to Help → Manage Licence → Get Free ISE WebPACK Licence. Follow the web process to get a licence file, then install it.
5. Download the Pynq-Z1 board files from Digilent:
[digilentinc.com/reference/programmable-logic/pynq-z1/start](https://www.digilentinc.com/reference/programmable-logic/pynq-z1/start). Copy the board_files folder into: Vivado/2022.2/data/boards/board_files/
6. Restart Vivado. When creating a new project, you should now see 'PYNQ-Z1' in the board selector.

Vivado Install Size

Vivado is large (~30GB with Zynq-7000 only). Install on an SSD with at least 50GB free. On a slow hard drive, synthesis can take 30+ minutes. On an SSD it typically takes 8-15 minutes.

1.3 Installing Icarus Verilog and GTKWave

These are used for simulation — you run them on your laptop, not on the Pynq. Simulation lets you debug Verilog in seconds rather than waiting 15 minutes for Vivado to build a bitstream.

OS	Installation Command
Ubuntu/Debian Linux	<code>sudo apt install iverilog gtkwave</code>
macOS (Homebrew)	<code>brew install icarus-verilog gtkwave</code>
Windows	Download installer from bleyer.org/icarus — includes GTKWave

Test the installation:

Test Icarus Verilog installation

```
# Create a test file
echo 'module test; initial begin $display("Icarus works"); $finish; end endmodule'
> test.v
# Compile and run
iverilog -o test test.v && vvp test
# Should print: Icarus works
```

1.4 Setting Up the Pynq

7. Connect Pynq-Z1 via micro-USB to your laptop and via HDMI to a monitor.
8. Set the boot jumper to SD (not JTAG). Insert the Pynq SD card.

9. Power on. Wait 30 seconds for boot.
10. Open a browser and go to 192.168.2.99 (USB connection) — you should see Jupyter notebook.
11. Default password: xilinx
12. Alternatively SSH: `ssh xilinx@192.168.2.99` (password: xilinx)
13. Transfer files with SCP: `scp myfile.bit xilinx@192.168.2.99:/home/xilinx/`

Pynq Network Options

USB: always works, fixed IP 192.168.2.99. Ethernet: connect to router, IP assigned by DHCP — check your router's device list. Ethernet is faster for large file transfers (bitstreams are ~10MB).

1.5 VS Code Verilog Extension

Install the 'Verilog-HDL/SystemVerilog/Bluespec SystemVerilog' extension by mshr-h. This gives syntax highlighting, error underlining, and signal navigation. Configure it to use Icarus Verilog as the linter:

VS Code settings.json

```
{
  "verilog.linting.linter": "iverilog",
  "verilog.linting.iverilog.arguments": "-Wall",
  "editor.tabSize": 2
}
```

SECTION 2 — VIVADO PROJECT SETUP

2. Vivado Project Setup — Step by Step

2.1 Creating the Project

14. Open Vivado. Click 'Create Project'.
15. Project name: em_fdt_visualiser. Location: choose a path on your SSD with no spaces.
16. Project type: RTL Project. Check 'Do not specify sources at this time'.
17. Default Part: click 'Boards' tab → search 'PYNQ-Z1' → select it → Next → Finish.

2.2 Building the Block Design

The Pynq uses a block design (IP Integrator) approach rather than pure Verilog at the top level. The block design connects the ARM CPU (PS — Processing System) to your custom FPGA logic (PL — Programmable Logic) via AXI buses.

18. In the Flow Navigator (left panel), click 'Create Block Design'. Name it: em_system.
19. In the block design canvas, right-click → Add IP. Search 'ZYNQ7' → add 'ZYNQ7 Processing System'.
20. Click 'Run Block Automation' (green banner at top). Select 'Apply Board Preset' for PYNQ-Z1. This configures DDR, UART, and other PS settings automatically.
21. Double-click the ZYNQ7 block → go to 'PS-PL Configuration' → enable: AXI GP0 (for AXI-Lite), AXI HP0 (for high-bandwidth DMA), FCLK_CLK0 at 100MHz.

2.2.1 Adding IP Blocks

Add each of these IP blocks by right-clicking in the canvas → Add IP:

IP Block	Configuration	Purpose
AXI GPIO	32-bit, dual channel	Register file for CPU↔FPGA parameter passing
XADC Wizard	Single channel, continuous, DRP interface	ADC for probe voltage measurement
AXI BRAM Controller ×3	32-bit, single port	CPU access to Ex, Ey, Bz field arrays
Block Memory Generator ×3	True dual port, 16-bit width, 65536 depth	Ex, Ey, Bz storage (256×256)

IP Block	Configuration	Purpose
AXI DMA	Simple mode, 32-bit	Move pixel data from FPGA to DDR for HDMI
Video Timing Controller	640×480 @ 60Hz preset	HDMI timing generation
AXI4-Stream to Video Out	RGB888	Convert pixel stream to HDMI signals
Clocking Wizard	Input 125MHz, output 100MHz + 200MHz	Generate stable clocks from board oscillator

22. After adding all IP, click 'Run Connection Automation' (green banner). This auto-connects AXI buses, clocks, and resets. Accept all defaults.
23. Manually connect: your custom FDTD top module's AXI slave port to the AXI Interconnect. Connect field BRAM ports to BRAM controllers. Connect pixel stream output to AXI DMA S2MM port.
24. Add your custom RTL source files: right-click in Sources panel → Add Sources → Add Files → select all your .v files.
25. Right-click the block design in Sources → Create HDL Wrapper. This wraps the block design so it can be the top-level module.

2.3 Adding Your Verilog to the Block Design

Custom modules connect to the block design via ports. The FDTD top module needs:

- An AXI-Lite slave port for register access (clock, reset, address, data lines)
- Connections to the three BRAM port B interfaces (read/write field data)
- An AXI-Stream master port for pixel output
- A clock and synchronous reset input

In Vivado, right-click the block design canvas → Add Module → select your `fdtd_top` Verilog module. It appears as a block. Connect its ports to the appropriate IPs.

2.4 Constraints File

Create a constraints file (.xdc) that tells Vivado which physical pins on the FPGA chip connect to which board signals. For the Pynq-Z1, Digilent provides a master XDC file — download it from their website and uncomment only the lines you use.

em_fdtd.xdc — key constraints

```
## HDMI output (from Pynq-Z1 master XDC)
set_property PACKAGE_PIN A15 [get_ports {hdmi_tx_p[2]}]
set_property IOSTANDARD TMDS_33 [get_ports {hdmi_tx_p[2]}]
## ... (full HDMI pins from master XDC)

## Arduino header — XADC analog inputs
```

```
## A0 = probe voltage (VP_0/VN_0)
set_property PACKAGE_PIN E17 [get_ports xadc_vp_in]
set_property PACKAGE_PIN D18 [get_ports xadc_vn_in]

## Arduino header - GPIO for touch panel and MUX
set_property PACKAGE_PIN T14 [get_ports {gpio_o[0]]}    ;# D0 - touch panel X+
set_property PACKAGE_PIN U12 [get_ports {gpio_o[1]]}    ;# D1 - touch panel Y+
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_o[0]]}

## Timing constraint
create_clock -period 10.000 -name sys_clk [get_ports sys_clk]
```

2.5 Synthesis, Implementation, and Bitstream

26. Flow Navigator → Run Synthesis. Wait 5-10 minutes. Fix any errors (usually port mismatch or unconnected signals).
27. Run Implementation. Wait 10-15 minutes. Check timing report — all paths must meet timing (no red in timing summary).
28. Generate Bitstream. Wait 5 minutes.
29. Export bitstream: File → Export → Export Bitstream File → save as em_fdt.d.bit.
30. Export hardware: File → Export → Export Hardware → include bitstream → save as em_fdt.d.xsa.
31. Copy em_fdt.d.bit to the Pynq: scp em_fdt.d.bit xilinx@192.168.2.99:/home/xilinx/

If Timing Fails

Reduce clock frequency from 100MHz to 75MHz in the Clocking Wizard and update the timing constraint. The FDTD arithmetic is the most likely critical path — add pipeline registers in the updater modules if needed.

SECTION 3 — WAVE INJECTION

3. Wave Injection — How It Works

3.1 What Wave Injection Means Physically

In a real EM experiment, a wave is generated by an oscillating current in an antenna or an oscillating voltage on an electrode. In your FDTD simulation, there is no real antenna — you create a wave by forcing one or more grid cells to follow a prescribed time-varying waveform. This is called source injection.

The physical analogy is exact: your conducting paper electrodes driven by the AD9833 are the physical source. In the simulation, grid cells at positions corresponding to those electrode positions are forced to oscillate — the simulation source mirrors the physical source. This is how the FDTD simulation and the physical measurement stay synchronised.

3.2 The Hard Source — Simplest Approach

3.2.1 What It Does

A hard source completely overrides the FDTD update equation at the source cell. Every time step, after the FDTD update runs across the whole grid, the source cell is simply set to a prescribed value regardless of what the FDTD equation computed:

$$Ex[isrc][jsrc] = A \times \sin(2\pi \times f_0 \times n \times \Delta t)$$

Where n is the time step counter, f_0 is the normalised source frequency, A is amplitude, and Δt is the time step.

3.2.2 Why It Causes Problems

A hard source reflects incoming waves. If a wave propagates outward from the source, reflects off a boundary or obstacle, and returns to the source cell, it hits the fixed value and reflects again — creating a non-physical double-reflection artefact. For a simple demo this is acceptable. For quantitatively accurate results, use a soft source.

3.2.3 Verilog Implementation

```
hard_source.v
module hard_source #(
    parameter NX    = 256,
    parameter NY    = 256,
    parameter FRAC = 13    // Q3.13 fixed point
) (
    input          clk,
    input          rst,
    input [31:0] time_step,    // global time step counter
```

```

input  [7:0]  src_x, src_y,      // source grid position
input  signed [17:0] amplitude, // source amplitude Q3.13
input  signed [17:0] f0,        // normalised frequency Q3.13
// BRAM write port for Ex
output reg      ex_we,          // write enable
output reg [15:0] ex_addr,      // address = x*NY + y
output reg signed [17:0] ex_din // data to write
);

// CORDIC IP computes sin(phase) – instantiate Xilinx CORDIC
wire signed [17:0] sin_out;
reg  signed [23:0] phase;      // Q11.13 phase accumulator

// Phase accumulator: increments by f0*2pi each step
// Represented as Q3.13: 2*pi = 6.283 -> 6.283 * 2^13 = 51472
localparam signed [23:0] TWO_PI = 24'd51472;

always @(posedge clk) begin
    if (rst) phase <= 0;
    else     phase <= phase + ((f0 * TWO_PI) >>> FRAC);
end

// CORDIC computes sin(phase) – see Section 3.4 for instantiation
cordic_sin u_cordic (.clk(clk), .phase(phase[17:0]), .sin_out(sin_out));

// Inject: override Ex at source position every time step
always @(posedge clk) begin
    ex_we    <= 1'b1;
    ex_addr <= src_x * NY + src_y;
    ex_din  <= (amplitude * sin_out) >>> FRAC;
end

endmodule

```

3.3 The Soft Source — Physically Correct

3.3.1 What It Does

A soft source adds the source waveform to the FDTD update result rather than replacing it:

$$Ex[i_{src}][j_{src}] += A \times \sin(2\pi \times f_0 \times n \times \Delta t)$$

This allows reflected waves to pass back through the source cell without artificial scattering. The source drives the field but does not block wave propagation through its location. This is the standard approach for antenna simulation.

3.3.2 Verilog — Modification to FDTD Engine

In the FDTD sequencer, after computing the Ex update for the source cell, add the source value before writing to BRAM:

soft source addition in ftdt_engine.v

```
// Inside the Ex update loop, after computing ex_new:
wire is_source_cell;
assign is_source_cell = (x_idx == src_x) && (y_idx == src_y);

wire signed [17:0] ex_final;
assign ex_final = is_source_cell ? (ex_new + source_value) : ex_new;

// Write ex_final to BRAM instead of ex_new
// source_value is computed by hard_source module (sin waveform)
// but here we ADD it rather than replacing
```

3.4 CORDIC for Sine Generation

The source waveform is a sine wave. Computing $\sin()$ in hardware without the Xilinx CORDIC IP would require a lookup table with interpolation. The CORDIC IP is much cleaner — configure it in Vivado as follows:

CORDIC Configuration	Setting
Functional Selection	Sin and Cos
Architectural Configuration	Parallel (fully pipelined)
Input Width	18 bits (matches Q3.13 phase)
Output Width	18 bits
Round Mode	Truncate
TLAST Behavior	Not Required
Flow Control	Blocking

The CORDIC input is a phase angle in the range $-\pi$ to $+\pi$, represented as a 18-bit fixed-point number. Your phase accumulator increments by $2\pi \times f_0 \times \Delta t$ each time step. When it exceeds π , wrap it back by subtracting 2π — this keeps the phase in range.

Phase accumulator and wrapping

```
// Phase accumulator (Q5.13 to handle range > 2*pi before wrapping)
reg signed [24:0] phase_accum;

// f0 * 2pi * dt in Q3.13:
// f0=0.05, dt=0.35, 2pi=6.283
// increment = 0.05 * 6.283 * 0.35 = 0.10995 -> 0.10995 * 8192 = 901
```

```

localparam signed [17:0] PHASE_INC = 18'd901;
localparam signed [24:0] PI        = 25'sd25736; // pi * 8192
localparam signed [24:0] TWO_PI    = 25'sd51472; // 2*pi * 8192

always @(posedge clk) begin
    if (rst)
        phase_accum <= 0;
    else begin
        phase_accum <= phase_accum + PHASE_INC;
        // Wrap at +pi
        if (phase_accum >= PI)
            phase_accum <= phase_accum - TWO_PI;
    end
end

// Feed to CORDIC (take lower 18 bits as Q3.13 phase)
assign cordic_phase_in = phase_accum[17:0];

```

3.5 Multiple Sources — Interference Patterns

To create interference patterns you need two or more sources. Each source has its own position, amplitude, and phase offset. The source injector module handles `N_SOURCES` sources using a small state machine that cycles through them.

multi_source_injector.v (simplified)

```

module multi_source_injector #(parameter N = 4) (
    input clk, rst,
    // Source parameters from AXI registers
    input [7:0]  src_x    [0:N-1],
    input [7:0]  src_y    [0:N-1],
    input signed [17:0] src_amp [0:N-1],
    input signed [17:0] src_phase [0:N-1], // phase offset in Q3.13
    input signed [17:0] src_f0,           // shared frequency
    input        src_en    [0:N-1],       // enable per source
    // Output: which cell to inject and how much
    output reg [15:0] inject_addr,
    output reg signed [17:0] inject_delta,
    output reg        inject_valid
);
    reg [1:0] src_idx; // which source we're injecting now
    reg signed [24:0] phase_accum [0:N-1]; // one accumulator per source
    wire signed [17:0] sin_out; // from shared CORDIC

    // Cycle through sources one per clock
    always @(posedge clk) begin
        if (rst) begin

```

```

src_idx <= 0;
inject_valid <= 0;
end else begin
    // Advance phase for current source
    phase_accum[src_idx] <= phase_accum[src_idx] + PHASE_INC +
src_phase[src_idx];
    // Output injection
    inject_addr <= src_x[src_idx] * 256 + src_y[src_idx];
    inject_delta <= src_en[src_idx] ? ((src_amp[src_idx] * sin_out) >>> 13) : 0;
    inject_valid <= src_en[src_idx];
    // Advance to next source
    src_idx <= (src_idx == N-1) ? 0 : src_idx + 1;
end
end
endmodule

```

3.6 Source Waveform Shapes

Different source waveforms demonstrate different physics:

Waveform	Formula	What It Shows
Continuous sine	$A \times \sin(2\pi f_0 n \Delta t)$	Steady-state interference patterns, standing waves
Gaussian pulse	$A \times \exp(-(n-n_0)^2/2\sigma^2) \times \sin(2\pi f_0 n \Delta t)$	Broadband pulse shows dispersion, all frequencies simultaneously
Ramp-up sine	$A \times (1 - \exp(-n/\tau)) \times \sin(2\pi f_0 n \Delta t)$	Smooth turn-on avoids startup transient artifacts
Single pulse	A if $n=n_0$, else 0	Impulse response — shows how field spreads from a point perturbation
Square wave	$A \times \text{sign}(\sin(2\pi f_0 n \Delta t))$	Rich harmonic content — shows dispersion clearly

Start with the ramp-up sine (avoids startup transients). Switch to Gaussian pulse for the most impressive demo — a single pulse propagates outward, reflects off the PML boundary (barely), and shows the full impulse response of the simulation domain.

3.7 How the Physical Electrode Connects to the Simulation Source

The AD9833 drives the physical electrode on the conducting paper at 1kHz. Simultaneously, the FPGA simulation runs its own source at normalised frequency f_0 . These are two independent

things — the physical electrode drives the real paper, the simulation source drives the virtual grid. They are connected conceptually (same position, same waveform shape) but not electrically.

The connection happens through boundary conditions: the probe measures the physical potential at the electrode position, and this value is written to the corresponding simulation grid cell. The physical measurement anchors the simulation to reality — the simulation field at the electrode must match what you physically measure.

SECTION 4 — MEASURING THE WAVE

4. Measuring the Wave

4.1 What You Are Actually Measuring

There are two completely separate measurement systems in this project:

- The conducting paper probe: measures the static or quasi-static potential $V(x,y)$ at any point on the paper surface — this is the Laplace solution, used as boundary condition for the FDTD simulation.
- The FDTD field arrays: the simulation computes E_x , E_y , B_z at every grid point — these are the wave fields, not directly measurable from the paper (the paper operates at too low a frequency to show wave propagation).

The FDTD simulation IS the wave measurement. The physical paper gives you the boundary conditions. The FPGA computes how an EM wave would behave given those boundary conditions. You display the computed wave fields in 3D.

4.2 Measuring the Physical Potential — The Probe Circuit

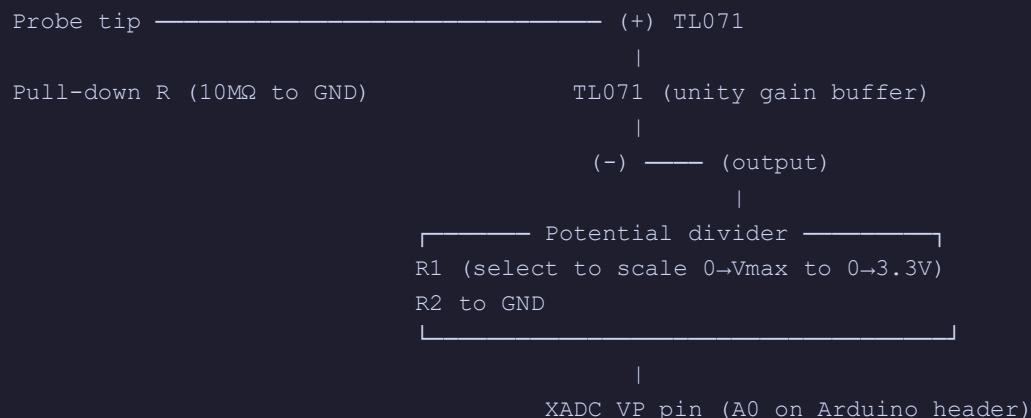
4.2.1 What the Probe Sees

The conducting paper with two electrodes has a potential distribution across its surface. The probe tip (a fine metal point) touches the paper at any (x,y) position and sits at the potential $V(x,y)$ of that point. This potential needs to be measured by the Pynq's XADC without disturbing the field — same loading problem as the water tray, same solution: a JFET input op-amp buffer.

4.2.2 The Complete Probe ADC Circuit

Probe ADC circuit (build on stripboard)

Conducting paper potential at probe tip:



Component values:

TL071: JFET input, $R_{in} > 10^{12} \Omega$ — does not load the paper
Pull-down $10M\Omega$: prevents floating input if probe lifted off paper
R1, R2: voltage divider to scale electrode drive voltage to 0-3.3V
If electrode drives at 0-5V: $R1 = 18k\Omega$, $R2 = 33k\Omega \rightarrow V_{out} = V_{in} \times 0.647$
Verify: $5V \times 0.647 = 3.24V < 3.3V \checkmark$

4.2.3 Using AC Drive with Lock-In on the Paper

If you use AC drive (1kHz from AD9833) on the conducting paper instead of DC, you get the same noise rejection benefits as with water. The probe then sees a 1kHz AC signal whose amplitude encodes the potential. The lock-in demodulation extracts this amplitude cleanly.

For the conducting paper with a handheld probe, lock-in is optional — paper has no electrolysis problem and DC is stable. But if lab noise is an issue, switch to AC drive and add the lock-in demodulation for the probe channel:

Python lock-in for probe measurement

```
import numpy as np
from pyng import MMIO

f0 = 1000          # drive frequency Hz
fs = 20000         # ADC sample rate Hz
N = 20            # samples per cycle

n = np.arange(N)
ref_cos = np.cos(2 * np.pi * f0 * n / fs)
ref_sin = np.sin(2 * np.pi * f0 * n / fs)

def measure_probe_voltage():
    # Collect N samples from XADC channel 0
    samples = np.array([read_xadc(channel=0) for _ in range(N)])
    # Lock-in demodulation
    I = np.mean(samples * ref_cos)
    Q = np.mean(samples * ref_sin)
    amplitude = 2.0 * np.sqrt(I**2 + Q**2)
    return amplitude # volts, noise-free
```

4.3 XY Position Sensing — Resistive Touch Panel

4.3.1 How a Resistive Touch Panel Works

A resistive touch panel has four layers: two transparent resistive sheets (ITO — Indium Tin Oxide) separated by a small air gap, with non-conductive spacer dots. When pressed, the two sheets make contact at the touch point.

To measure X position: apply voltage across the top sheet left-to-right, measure voltage at the contact point on the bottom sheet. The voltage is proportional to the X position — it is a simple resistive voltage divider. To measure Y position: apply voltage across the bottom sheet top-to-bottom, measure on the top sheet.

The panel has four connections: X+, X-, Y+, Y-. You drive two and measure two alternately.

4.3.2 Connecting to the Pynq

Touch panel connections to Pynq Arduino header

Touch panel pin → Pynq Arduino header

```
X+ → D2 (GPIO output — drive high or float)
X- → D3 (GPIO output — drive low or float)
Y+ → A1 (XADC channel 1 — measure X position voltage)
Y- → D4 (GPIO output — drive high or float)
```

For Y measurement:

```
Y+ → drive high (D4 high)
Y- → drive low (implicit via resistor to GND)
X+ → A2 (XADC channel 2 — measure Y position voltage)
X- → float
```

Alternate between X and Y measurement at ~100Hz
(fast enough — probe moves much slower than 100Hz)

4.3.3 Python Reading Code

touch_panel.py

```
from pynq import GPIO, MMIO
import time

# GPIO pins for driving touch panel
x_plus = GPIO(GPIO.get_gpio_pin(0), 'out') # D2
x_minus = GPIO(GPIO.get_gpio_pin(1), 'out') # D3
y_plus = GPIO(GPIO.get_gpio_pin(2), 'out') # D4

def read_x():
    # Drive X axis, read on Y+
    x_plus.write(1)
    x_minus.write(0)
    y_plus.write(0) # float Y
    time.sleep(0.001) # 1ms settle
    raw = read_xadc(channel=1) # 0 to 3.3V
```

```

        return raw / 3.3                # normalise 0 to 1.0

def read_y():
    # Drive Y axis, read on X+
    y_plus.write(1)
    x_plus.write(0)    # float X
    x_minus.write(0)
    time.sleep(0.001)
    raw = read_xadc(channel=2)
    return raw / 3.3

def read_probe_position():
    x = read_x()
    y = read_y()
    v = measure_probe_voltage()    # from lock-in or direct ADC
    return x, y, v    # all in range 0.0 to 1.0 (x,y) and volts (v)

# Map to grid indices
def to_grid(x_norm, y_norm):
    ix = int(x_norm * 255)    # 0 to 255
    iy = int(y_norm * 255)    # 0 to 255
    return ix, iy

```

4.4 Reading the XADC from Python

The Pynq's XADC Wizard IP exposes its outputs as memory-mapped registers. The Python MMIO class reads them directly:

```

xadc_read.py
from pynq import Overlay, MMIO

# Load your bitstream first
ol = Overlay('/home/xilinx/em_fdt.bit')

# Access XADC via its AXI-Lite register interface
# Base address from Vivado address editor (typically 0x43C00000)
xadc = MMIO(0x43C00000, 0x10000)

def read_xadc(channel):
    # XADC status register addresses:
    # Channel 0 (VP/VN) = offset 0x200
    # Channel 1 (VAUX1) = offset 0x210
    # Channel 2 (VAUX9) = offset 0x218
    offsets = {0: 0x200, 1: 0x210, 2: 0x218}
    raw = xadc.read(offsets[channel])

```

```

    # Raw value is 16-bit left-justified: bits [15:4] are the 12-bit result
    adc_value = (raw >> 4) & 0xFFF    # 0 to 4095
    voltage = adc_value * 3.3 / 4096  # convert to volts
    return voltage

# Test:
print(f'Probe voltage: {read_xadc(0):.3f} V')
print(f'Touch X:      {read_xadc(1):.3f} V')
print(f'Touch Y:      {read_xadc(2):.3f} V')

```

4.5 Reading the FDTD Field Arrays from Python

The field BRAMs are accessible from Python via the AXI BRAM controller. This lets you read the current state of the simulation at any time — useful for debugging and for the comparison display:

```

read_field.py
import numpy as np
from pynq import MMIO

# BRAM base addresses from Vivado address editor
EX_BRAM_BASE = 0x40000000
EY_BRAM_BASE = 0x40010000
BZ_BRAM_BASE = 0x40020000
NX, NY = 256, 256

ex_mmio = MMIO(EX_BRAM_BASE, NX*NY*4)  # 4 bytes per word
ey_mmio = MMIO(EY_BRAM_BASE, NX*NY*4)
bz_mmio = MMIO(BZ_BRAM_BASE, NX*NY*4)

def read_field_ex():
    raw = np.array([ex_mmio.read(i*4) for i in range(NX*NY)], dtype=np.int32)
    # Convert from Q3.13 fixed-point to float
    return raw.astype(np.float32) / 8192.0  # divide by 2^13

def read_field_bz():
    raw = np.array([bz_mmio.read(i*4) for i in range(NX*NY)], dtype=np.int32)
    return raw.astype(np.float32) / 8192.0

# Read and reshape to 2D grid
ex_field = read_field_ex().reshape(NX, NY)
print(f'Ex range: {ex_field.min():.4f} to {ex_field.max():.4f}')

```

BRAM Read Speed

Reading 65,536 values one by one over AXI-Lite is slow (~0.5 seconds for one field). For the demo display, the FPGA reads the BRAM directly and renders — the CPU only reads the field for debugging or for the comparison plot. Use DMA for fast bulk transfers if you need the full field in Python at runtime.

4.6 Writing Boundary Conditions to the FPGA

The probe measurement must be written to the FPGA as a boundary condition — telling the FDTD engine which cells are fixed and at what value. This is done via AXI-Lite registers:

boundary_conditions.py

```
from pynq import MMIO

# AXI GPIO base address from Vivado address editor
REGS_BASE = 0x41200000
regs = MMIO(REGS_BASE, 0x1000)

# AXI register map (32-bit each, 4 bytes apart):
# Reg 0: rotation theta (16-bit)
# Reg 1: rotation phi (16-bit)
# Reg 2: source 1 X position (8-bit)
# Reg 3: source 1 Y position (8-bit)
# Reg 4: source 1 amplitude (18-bit Q3.13)
# Reg 5: source 2 X position
# Reg 6: source 2 Y position
# Reg 7: source 2 amplitude
# Reg 8: boundary cell X (from probe)
# Reg 9: boundary cell Y (from probe)
# Reg 10: boundary cell value (Q3.13)
# Reg 11: boundary write strobe (pulse high to latch)
# Reg 12: display mode (0=Ex, 1=Bz, 2=Poynting, 3=all)

def write_register(reg_num, value):
    regs.write(reg_num * 4, int(value) & 0xFFFFFFFF)

def set_boundary_condition(x, y, voltage_normalised):
    # Convert voltage (0.0-1.0 normalised) to Q3.13 fixed point
    fixed = int(voltage_normalised * 8192) # * 2^13
    write_register(8, x)
    write_register(9, y)
    write_register(10, fixed)
    write_register(11, 1) # strobe — FPGA latches new BC
    write_register(11, 0) # clear strobe

def set_source(src_idx, x, y, amplitude):
```

```
base = 2 + src_idx * 3
write_register(base, x)
write_register(base+1, y)
write_register(base+2, int(amplitude * 8192))
```

SECTION 5 — COMPLETE VERILOG

5. Complete Verilog — Every Module

5.1 Top-Level Module

fddt_top.v

```
module fddt_top (
    input        sys_clk,
    input        sys_rst_n,
    // AXI-Lite slave (from PS)
    input  [31:0] s_axi_awaddr, s_axi_wdata,
    input        s_axi_awvalid, s_axi_wvalid, s_axi_bready,
    output       s_axi_awready, s_axi_wready, s_axi_bvalid,
    input  [31:0] s_axi_araddr,
    input        s_axi_arvalid, s_axi_rready,
    output [31:0] s_axi_rdata,
    output       s_axi_arready, s_axi_rvalid,
    // AXI-Stream master (pixel output to VDMA)
    output [23:0] m_axis_tdata, // RGB888
    output       m_axis_tvalid,
    output       m_axis_tlast, // end of line
    output       m_axis_tuser, // start of frame
    input        m_axis_tready,
    // XADC
    input        xadc_vp, xadc_vn
);
    wire clk = sys_clk;
    wire rst = ~sys_rst_n;

    // AXI register file - 24 registers
    wire [31:0] regs [0:23];
    axi_lite_regs u_regs (.clk(clk), .rst(rst), /* AXI ports */, .regs_o(regs));

    // Parse registers
    wire [15:0] theta = regs[0][15:0];
    wire [15:0] phi   = regs[1][15:0];
    wire [7:0]  src1_x = regs[2][7:0];
    wire [7:0]  src1_y = regs[3][7:0];
    wire signed [17:0] src1_amp = regs[4][17:0];
    wire [7:0]  src2_x = regs[5][7:0];
    wire [7:0]  src2_y = regs[6][7:0];
    wire signed [17:0] src2_amp = regs[7][17:0];
```

```

wire [7:0]  bc_x      = regs[8][7:0];
wire [7:0]  bc_y      = regs[9][7:0];
wire signed [17:0] bc_val = regs[10][17:0];
wire        bc_strobe = regs[11][0];
wire [1:0]  disp_mode = regs[12][1:0];

// FDTD engine - computes Ex, Ey, Bz
wire [15:0] ex_rd_addr, ey_rd_addr, bz_rd_addr;
wire signed [17:0] ex_rd_data, ey_rd_data, bz_rd_data;
fdtd_engine u_fdttd (
    .clk(clk), .rst(rst),
    .src1_x(src1_x), .src1_y(src1_y), .src1_amp(src1_amp),
    .src2_x(src2_x), .src2_y(src2_y), .src2_amp(src2_amp),
    .bc_x(bc_x), .bc_y(bc_y), .bc_val(bc_val), .bc_strobe(bc_strobe),
    // BRAM ports A (engine write/read), B (renderer read)
    .ex_bram_b_addr(ex_rd_addr), .ex_bram_b_dout(ex_rd_data),
    .ey_bram_b_addr(ey_rd_addr), .ey_bram_b_dout(ey_rd_data),
    .bz_bram_b_addr(bz_rd_addr), .bz_bram_b_dout(bz_rd_data)
);

// 3D renderer
renderer_3d u_renderer (
    .clk(clk), .rst(rst),
    .theta(theta), .phi(phi), .disp_mode(disp_mode),
    .ex_addr(ex_rd_addr), .ex_data(ex_rd_data),
    .ey_addr(ey_rd_addr), .ey_data(ey_rd_data),
    .bz_addr(bz_rd_addr), .bz_data(bz_rd_data),
    .m_axis_tdata(m_axis_tdata), .m_axis_tvalid(m_axis_tvalid),
    .m_axis_tlast(m_axis_tlast), .m_axis_tuser(m_axis_tuser),
    .m_axis_tready(m_axis_tready)
);
endmodule

```

5.2 FDTD Engine — Core Sequencer

fdtd_engine.v

```

module fdtd_engine #(
    parameter NX = 256,
    parameter NY = 256,
    parameter PML_WIDTH = 8,           // PML border thickness in cells
    parameter signed [17:0] CB_B = 18'sd2867, //  $\Delta t/\Delta x = 0.35$  in Q3.13
    parameter signed [17:0] CB_E = 18'sd717   //  $c^2 \times \Delta t/\Delta x = 0.0875$  in Q3.13
) (
    input clk, rst,
    // Source parameters

```

```

input [7:0] src1_x, src1_y, src2_x, src2_y,
input signed [17:0] src1_amp, src2_amp,
// Boundary condition
input [7:0] bc_x, bc_y,
input signed [17:0] bc_val,
input bc_strobe,
// BRAM port B for renderer read
input [15:0] ex_bram_b_addr, ey_bram_b_addr, bz_bram_b_addr,
output signed [17:0] ex_bram_b_dout, ey_bram_b_dout, bz_bram_b_dout
);

// — BRAM instantiation —————
reg [15:0] bram_addr_a;
reg signed [17:0] ex_din, ey_din, bz_din;
reg ex_we, ey_we, bz_we;
wire signed [17:0] ex_dout, ey_dout, bz_dout;

bram_18x65536 u_ex_bram (.clk(clk),
    .a_addr(bram_addr_a), .a_din(ex_din), .a_we(ex_we), .a_dout(ex_dout),
    .b_addr(ex_bram_b_addr), .b_dout(ex_bram_b_dout));
bram_18x65536 u_ey_bram (.clk(clk),
    .a_addr(bram_addr_a), .a_din(ey_din), .a_we(ey_we), .a_dout(ey_dout),
    .b_addr(ey_bram_b_addr), .b_dout(ey_bram_b_dout));
bram_18x65536 u_bz_bram (.clk(clk),
    .a_addr(bram_addr_a), .a_din(bz_din), .a_we(bz_we), .a_dout(bz_dout),
    .b_addr(bz_bram_b_addr), .b_dout(bz_bram_b_dout));

// — State machine —————
localparam S_IDLE=0, S_UPD_BZ=1, S_INJ_SRC=2, S_PML_BZ=3,
           S_UPD_EX=4, S_UPD_EY=5, S_PML_E=6, S_DONE=7;
reg [2:0] state;
reg [15:0] cell_x, cell_y; // current cell being processed
reg [31:0] time_step;

// Source waveforms
wire signed [17:0] src1_wave, src2_wave;
source_gen u_src1 (.clk(clk), .rst(rst), .time_step(time_step),
    .amplitude(src1_amp), .wave_out(src1_wave));
source_gen u_src2 (.clk(clk), .rst(rst), .time_step(time_step),
    .amplitude(src2_amp), .wave_out(src2_wave));

// PML coefficient LUT (8 values for 8-cell boundary)
reg signed [17:0] pml_coeff [0:7];
initial begin
    // Exponential decay: 0.90^1 to 0.90^8 in Q3.13
    pml_coeff[0] = 18'sd7373; // 0.90 * 8192
    pml_coeff[1] = 18'sd6636; // 0.81 * 8192
    pml_coeff[2] = 18'sd5972; // 0.729 * 8192

```

```

    pml_coeff[3] = 18'sd5375; // 0.656 * 8192
    pml_coeff[4] = 18'sd4837; // 0.590 * 8192
    pml_coeff[5] = 18'sd4354; // 0.531 * 8192
    pml_coeff[6] = 18'sd3918; // 0.478 * 8192
    pml_coeff[7] = 18'sd3526; // 0.430 * 8192
end

// Boundary condition storage (store up to 16 fixed cells)
reg [7:0] bc_cells_x [0:15];
reg [7:0] bc_cells_y [0:15];
reg signed [17:0] bc_cells_v [0:15];
reg [3:0] bc_count;

// Latch new boundary conditions when strobe fires
always @(posedge clk) begin
    if (bc_strobe && bc_count < 15) begin
        bc_cells_x[bc_count] <= bc_x;
        bc_cells_y[bc_count] <= bc_y;
        bc_cells_v[bc_count] <= bc_val;
        bc_count <= bc_count + 1;
    end
end

// Check if current cell is a boundary cell
wire is_bc;
wire signed [17:0] bc_field_val;
bc_checker u_bc (.x(cell_x[7:0]), .y(cell_y[7:0]),
    .bc_x(bc_cells_x), .bc_y(bc_cells_y), .bc_v(bc_cells_v),
    .bc_count(bc_count), .is_bc(is_bc), .bc_val_out(bc_field_val));

// — Main sequencer —————
always @(posedge clk) begin
    if (rst) begin
        state <= S_IDLE; cell_x <= 0; cell_y <= 0; time_step <= 0;
        ex_we <= 0; ey_we <= 0; bz_we <= 0;
    end else begin
        case (state)

            S_IDLE: begin
                // Start a new FDTD time step
                cell_x <= 0; cell_y <= 0;
                state <= S_UPD_BZ;
            end

            S_UPD_BZ: begin
                // Read  $E_x(x,y+1) - E_x(x,y) - E_y(x+1,y) + E_y(x,y)$  from BRAM
                // (This takes multiple clock cycles - simplified here)
            end
        endcase
    end
end

```

```

    // See bz_updater module for detailed pipeline
    bz_we <= ~is_bc; // don't update boundary cells
    bz_din <= is_bc ? bc_field_val : bz_new_value;
    bram_addr_a <= cell_x * NY + cell_y;
    // Advance cell counter
    if (cell_y == NY-1) begin
        cell_y <= 0;
        if (cell_x == NX-1) begin cell_x <= 0; state <= S_INJ_SRC; end
        else cell_x <= cell_x + 1;
    end else cell_y <= cell_y + 1;
end

S_INJ_SRC: begin
    // Add source waveforms at source positions (soft source)
    ex_we <= 1; ey_we <= 1;
    // src 1
    bram_addr_a <= src1_x * NY + src1_y;
    ex_din <= ex_dout + src1_wave; // soft add
    // src 2 (next clock cycle)
    // ... handled by sub-state
    time_step <= time_step + 1;
    state <= S_UPD_EX;
end

S_UPD_EX: begin
    // Ex update loop - similar structure to BZ
    ex_we <= ~is_bc;
    ex_din <= is_bc ? bc_field_val : ex_new_value;
    bram_addr_a <= cell_x * NY + cell_y;
    if (cell_y == NY-1) begin
        cell_y <= 0;
        if (cell_x == NX-1) begin cell_x <= 0; state <= S_UPD_EY; end
        else cell_x <= cell_x + 1;
    end else cell_y <= cell_y + 1;
end

S_UPD_EY: begin
    // Ey update loop
    ey_we <= ~is_bc;
    ey_din <= is_bc ? bc_field_val : ey_new_value;
    bram_addr_a <= cell_x * NY + cell_y;
    if (cell_y == NY-1) begin
        cell_y <= 0;
        if (cell_x == NX-1) begin
            cell_x <= 0;
            state <= S_IDLE; // done - next frame
        end else cell_x <= cell_x + 1;
    end else cell_y <= cell_y + 1;
end

```

```

        end

        default: state <= S_IDLE;
    endcase
end
end
endmodule

```

5.3 The Update Arithmetic Modules

bz_updater.v — full pipelined implementation

```

// Computes new Bz at (i,j) from surrounding Ex and Ey
// Pipeline depth: 3 clock cycles
// Input: reads from BRAM must be presented externally
module bz_updater (
    input clk,
    input signed [17:0] bz_cur,    // Bz(i,j)
    input signed [17:0] ex_j,      // Ex(i, j) - at Yee position (i+0.5, j)
    input signed [17:0] ex_j1,     // Ex(i, j+1) - at Yee position (i+0.5, j+1)
    input signed [17:0] ey_i,      // Ey(i, j) - at Yee position (i, j+0.5)
    input signed [17:0] ey_i1,     // Ey(i+1, j) - at Yee position (i+1, j+0.5)
    input signed [17:0] CB_B,      // = Δt/Δx in Q3.13
    output reg signed [17:0] bz_new
);
    // Stage 1: compute curl_E = (Ex_j1 - Ex_j) - (Ey_i1 - Ey_i)
    reg signed [18:0] curl_E; // one extra bit for overflow
    always @(posedge clk)
        curl_E <= (ex_j1 - ex_j) - (ey_i1 - ey_i);

    // Stage 2: multiply curl_E by CB_B
    reg signed [36:0] product;
    always @(posedge clk)
        product <= CB_B * curl_E;

    // Stage 3: truncate and update
    always @(posedge clk)
        bz_new <= bz_cur - product[30:13]; // >> 13 for Q3.13 fixed point
endmodule

// -----
module ex_updater (
    input clk,
    input signed [17:0] ex_cur,
    input signed [17:0] bz_j,      // Bz(i, j)
    input signed [17:0] bz_jm1,    // Bz(i, j-1)

```

```

input  signed [17:0] CB_E,      // =  $c^2 \times \Delta t / \Delta x$  in Q3.13
output reg signed [17:0] ex_new
);
reg signed [18:0] diff;
reg signed [36:0] product;
always @(posedge clk) diff    <= bz_j - bz_jm1;
always @(posedge clk) product <= CB_E * diff;
always @(posedge clk) ex_new  <= ex_cur + product[30:13];
endmodule

module ey_updater (
input  clk,
input  signed [17:0] ey_cur,
input  signed [17:0] bz_i,      // Bz(i, j)
input  signed [17:0] bz_im1,    // Bz(i-1, j)
input  signed [17:0] CB_E,
output reg signed [17:0] ey_new
);
reg signed [18:0] diff;
reg signed [36:0] product;
always @(posedge clk) diff    <= bz_i - bz_im1;
always @(posedge clk) product <= CB_E * diff;
always @(posedge clk) ey_new  <= ey_cur - product[30:13]; // note minus
endmodule

```

5.4 Poynting Vector Calculator

```

poynting_calc.v
// Computes  $S_x = E_y B_z / \mu_0$ ,  $S_y = -E_x B_z / \mu_0$  per pixel
// In normalised units  $\mu_0$  is absorbed into CB_E/CB_B coefficients
//  $|S| = \sqrt{S_x^2 + S_y^2}$  computed by CORDIC
module poynting_calc (
input  clk,
input  signed [17:0] ex, ey, bz,
output reg signed [17:0] s_mag // |S| magnitude
);
// Stage 1: compute  $S_x = E_y B_z$ ,  $S_y = -E_x B_z$ 
reg signed [35:0] sx_raw, sy_raw;
always @(posedge clk) begin
    sx_raw <= ey * bz;
    sy_raw <= -ex * bz;
end

// Stage 2: truncate to Q3.13
reg signed [17:0] sx, sy;

```

```
always @(posedge clk) begin
    sx <= sx_raw[30:13];
    sy <= sy_raw[30:13];
end

// Stage 3: |S| = sqrt(sx^2 + sy^2) via CORDIC
// Xilinx CORDIC IP in vector mode: input (sx,sy), output magnitude
cordic_magnitude u_cordic (
    .clk(clk),
    .x_in(sx), .y_in(sy),
    .magnitude(s_mag)
);
endmodule
```

SECTION 6 — COMPLETE PYTHON CONTROL

6. Complete Python Control Layer

6.1 Main Control Loop

main.py — complete control loop

```
#!/usr/bin/env python3
"""
Main control loop for FDTD EM Field Visualiser
Runs on Pynq at 50Hz
"""
from pynq import Overlay, MMIO, GPIO
import numpy as np
import time

# — Load bitstream —————
print('Loading bitstream...')
ol = Overlay('/home/xilinx/em_fdttd.bit')
print('Loaded.')

# — Memory map —————
REGS_BASE = 0x41200000    # AXI GPIO — get from Vivado address editor
XADC_BASE = 0x43C00000    # XADC Wizard
regs = MMIO(REGS_BASE, 0x1000)
xadc = MMIO(XADC_BASE, 0x10000)

# — Helper functions —————
def wreg(n, v):    regs.write(n*4, int(v) & 0xFFFFFFFF)
def rreg(n):       return regs.read(n*4)

def read_xadc(ch):
    offsets = {0:0x200, 1:0x210, 2:0x218, 3:0x220}
    raw = (xadc.read(offsets[ch]) >> 4) & 0xFFF
    return raw * 3.3 / 4096

def float_to_q313(f):
    """Convert float to Q3.13 fixed point integer"""
    return int(np.clip(f, -4.0, 3.999) * 8192)

# — Touch panel GPIO —————
xp_drive = GPIO(GPIO.get_gpio_pin(0), 'out') # X+ drive
```

```

xm_drive = GPIO(GPIO.get_gpio_pin(1), 'out') # X- drive
yp_drive = GPIO(GPIO.get_gpio_pin(2), 'out') # Y+ drive

def read_touch_xy():
    # Measure X: drive X axis, read on Y+
    xp_drive.write(1); xm_drive.write(0); yp_drive.write(0)
    time.sleep(0.001)
    x_norm = read_xadc(1) / 3.3
    # Measure Y: drive Y axis, read on X+
    yp_drive.write(1); xp_drive.write(0); xm_drive.write(0)
    time.sleep(0.001)
    y_norm = read_xadc(2) / 3.3
    return np.clip(x_norm,0,1), np.clip(y_norm,0,1)

# — Initialise FPGA registers —————
wreg(0, 32767) # theta = 45 degrees (initial rotation)
wreg(1, 16383) # phi = 30 degrees (initial tilt)
wreg(2, 128) # source 1 X = centre
wreg(3, 100) # source 1 Y
wreg(4, float_to_q313(1.0)) # source 1 amplitude
wreg(5, 128) # source 2 X
wreg(6, 156) # source 2 Y
wreg(7, float_to_q313(1.0)) # source 2 amplitude
wreg(12, 0) # display mode: Ex field

print('FPGA initialised. Starting control loop...')

# — Main loop —————
loop_count = 0
while True:
    t_start = time.time()

    # Read rotation potentiometers (on Arduino analog pins A3, A4, A5)
    theta_v = read_xadc(3) # 0 to 3.3V
    phi_v = read_xadc(4)
    zoom_v = read_xadc(5)

    # Map to register values
    theta = int(theta_v / 3.3 * 65535)
    phi = int(phi_v / 3.3 * 32767)

    wreg(0, theta)
    wreg(1, phi)

    # Read probe position and voltage
    x_norm, y_norm = read_touch_xy()
    v_probe = read_xadc(0) # probe voltage from conducting paper

```

```
# Map to grid coordinates
ix = int(x_norm * 255)
iy = int(y_norm * 255)

# Write boundary condition to FPGA
# Normalise voltage: 0V = 0.0, 5V = 1.0
v_norm = v_probe / 5.0
wreg(8, ix)
wreg(9, iy)
wreg(10, float_to_q313(v_norm * 2.0 - 1.0)) # centre around 0
wreg(11, 1) # strobe
wreg(11, 0) # clear strobe

# Mode button (GPIO digital input on D5)
# ... read GPIO, cycle display mode

loop_count += 1
if loop_count % 50 == 0:
    print(f'Probe: ({ix:3d},{iy:3d}) V={v_probe:.2f}V theta={theta:5d}')

# 50Hz loop rate
elapsed = time.time() - t_start
sleep_time = max(0, 0.02 - elapsed)
time.sleep(sleep_time)
```

SECTION 7 — TESTING AND DEBUGGING

7. Testing and Debugging Strategy

7.1 Simulation-First Workflow

Never load untested Verilog to the FPGA. Every module must pass simulation first. The workflow for each module is:

simulation_workflow.sh

```
# Step 1: Write the module (e.g. bz_updater.v)
# Step 2: Write a testbench (bz_updater_tb.v)
# Step 3: Compile with Icarus
iverilog -o bz_test bz_updater.v bz_updater_tb.v
# Step 4: Run simulation, generate waveform dump
vvp bz_test
# Step 5: View waveforms
gtkwave dump.vcd &
```

7.2 FDTD Engine Testbench

fddt_engine_tb.v — key test cases

```
`timescale 1ns/1ps
module fddt_engine_tb;
    reg clk=0, rst=1;
    always #5 clk = ~clk; // 100MHz

    // DUT
    fddt_engine uut (/* ports */);

    // Test sequence
    initial begin
        $dumpfile("fddt.vcd");
        $dumpvars(0, fddt_engine_tb);

        // Reset for 10 cycles
        repeat(10) @(posedge clk);
        rst = 0;

        // Run for 10000 time steps
        repeat(10000 * 270000) @(posedge clk);
    end
endmodule
```

```

// Check 1: field should not blow up
// (read BRAM and verify |Ex| < 2.0)

// Check 2: field at source should be oscillating
// (sample Ex at source position over 100 steps, verify sine-like)

$display("Simulation complete");
$finish;
end

// Monitor for overflow
always @(posedge clk) begin
    if (uut.ex_din > 18'sh1FFFF || uut.ex_din < -18'sh20000)
        $display("WARNING: Ex overflow at t=%0t", $time);
    end
endmodule

```

7.3 Debug Checklist

Problem	Likely Cause	Fix
Fields blow up immediately	CFL violated — Δt too large	Reduce Δt or c_eff . Check CB_B constant value.
Wave does not propagate	CB_E wrong sign or zero	Check $ex_updater$: should ADD (not subtract) $CB_E \times (Bz_j - Bz_{[j-1]})$
Wave reflects off boundaries	PML not active or coefficient too high	Check PML address range. Reduce decay to 0.95 per cell.
Source does not oscillate	CORDIC not outputting correctly	Check phase accumulator range — must stay in $[-\pi, +\pi]$
Display all one colour	Renderer not reading BRAM correctly	Check BRAM port B address. Print BRAM values from Python.
Boundary condition ignored	bc_strobe not pulsing correctly	Add $\$display$ in Verilog to confirm strobe seen. Check strobe clears.
XADC reads always 0	Wrong register offset	Check XADC status register address — use Vivado address editor.
Touch panel reads wrong position	X and Y drives swapped	Swap xp/yp drive signals in Python. Re-calibrate.
Timing fails in Vivado	Critical path too long	Add pipeline register in updater multiply stage. Reduce clock to 75MHz.

Problem	Likely Cause	Fix
Python import errors on Pynq	Wrong pynq version	pip3 install --upgrade pynq. Check overlay path is correct.

7.4 Verifying FDTD is Correct

Before the demo you must verify the simulation is physically correct. Three checks:

32. Energy conservation: with no source and no PML, run for 1000 steps. The total field energy $\sum(E_x^2 + E_y^2 + B_z^2)$ should not change. Any change indicates a bug in the update equations.
33. Wave speed: inject a single pulse. Measure how many time steps it takes to travel from the source to a point 50 cells away. Speed should be $c_{\text{eff}} \times \Delta x / \Delta t = 0.5$ cells/step. If 50 cells takes 100 steps: speed = 0.5 ✓
34. Wavelength: with continuous source at $f_0 = 0.05$, wavelength should be $c_{\text{eff}} / f_0 = 0.5 / 0.05 = 10$ cells. Count the distance between wavefronts visible on screen — should be 10 pixels.

SECTION 8 — PHYSICAL CONNECTIONS

8. Physical Connections on the Pynq

8.1 Pynq-Z1 Header Pinout — What Goes Where

Pynq Pin	Header	Connected To	Signal
A0 (VP/VN)	Arduino analog	Probe ADC circuit output	V_probe — conducting paper potential
A1	Arduino analog	Touch panel Y+ pin	Touch X position voltage
A2	Arduino analog	Touch panel X+ pin	Touch Y position voltage
A3	Arduino analog	Rotation potentiometer 1	FDTD 3D view theta (azimuth)
A4	Arduino analog	Rotation potentiometer 2	FDTD 3D view phi (elevation)
A5	Arduino analog	Zoom potentiometer	3D perspective zoom
D0	Arduino digital	Touch panel X+ drive	GPIO output — drive high for X measurement
D1	Arduino digital	Touch panel X- drive	GPIO output — drive low for X measurement
D2	Arduino digital	Touch panel Y+ drive	GPIO output — drive high for Y measurement
D3	Arduino digital	Mode button	GPIO input — cycle display mode
D4	Arduino digital	Source 2 enable switch	GPIO input — enable second source
PMODA pin 1	Pmod	AD9833 SPI MOSI	Sine wave frequency programming
PMODA pin 2	Pmod	AD9833 SPI CLK	SPI clock
PMODA pin 3	Pmod	AD9833 SPI CS	SPI chip select
HDMI out	HDMI	Monitor	3D field visualisation display

8.2 AD9833 SPI Connection

ad9833_spi.py — program sine wave frequency

```
from pynq.lib import MicroblazeLibrary
from pynq import Overlay

# Access SPI on Pmod connector
# Pynq provides SPI via the Arduino SPI pins or Pmod
import spidev

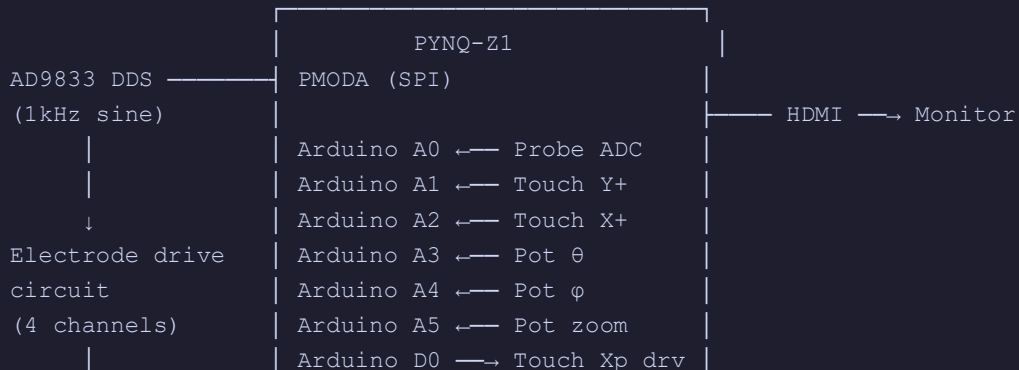
spi = spidev.SpiDev()
spi.open(1, 0) # bus 1, device 0
spi.max_speed_hz = 1000000
spi.mode = 0b10 # SPI mode 2 for AD9833

def ad9833_set_frequency(freq_hz, mclk=25e6):
    """Program AD9833 to output sine wave at freq_hz"""
    freq_word = int(freq_hz * (2**28) / mclk)
    msb = (freq_word >> 14) & 0x3FFF
    lsb = freq_word & 0x3FFF
    # Write control: reset, then freq LSB, MSB, then run
    def writel6(val):
        spi.xfer2([(val >> 8) & 0xFF, val & 0xFF])
    writel6(0x2100) # reset
    writel6(0x4000 | lsb) # FREQ0 LSB
    writel6(0x4000 | msb) # FREQ0 MSB
    writel6(0x2000) # release reset, sine output

ad9833_set_frequency(1000) # 1kHz
print('AD9833 set to 1kHz')
```

8.3 Complete Wiring Diagram

Physical wiring summary



↓ Conducting paper Resistive touch panel (on top of paper) Probe tip (user holds)	Arduino D1 → Touch Xm drv Arduino D2 → Touch Yp drv Arduino D3 ← Mode button
--	--

Everything Fits

All signals use the Arduino header (6 analog + 5 digital) and one Pmod connector (SPI for AD9833). No custom PCB is needed for the Pynq connections — use jumper wires from a small breadboard to the headers.

9. Quick Reference — Key Constants

Constant	Value	Fixed-Point Q3.13	Physical Meaning
$\Delta x = \Delta y$	1 (normalised)	8192	Grid cell size
Δt	0.35	2867	Time step
c_eff	0.5	4096	Effective wave speed ($c/\sqrt{\epsilon_r}$)
CB_B (Bz coeff)	0.35	2867	$= \Delta t/\Delta x$
CB_E (Ex,Ey coeff)	0.0875	717	$= c_eff^2 \times \Delta t/\Delta x$
f_0 (source freq)	0.05	410	Normalised frequency
λ (wavelength)	10 cells	—	$= c_eff/f_0 = 0.5/0.05$
Phase increment	0.10996/step	901	$= f_0 \times 2\pi \times \Delta t$
PML decay/cell	0.90	7373	Exponential boundary absorption
Grid size	256×256	—	65,536 cells total

How to Verify Constants

Check CB_B: a wave should travel exactly $c_eff = 0.5$ cells per time step. After 100 time steps with a pulse source, the wavefront should be at cell 50. Check CB_E: the Ex amplitude far from the source should equal the Bz amplitude (equal E and B in far field). If CB_E is wrong these will be unequal.